

Phase 7 - Local Storage

Fesche Task Manager | Notes | Calendar

A study guide for YOUR project code. Read this with localStorage.js open.

1. What this phase is

Before Phase 7, tasks, notes and events lived only in JavaScript arrays. Refresh the page and they disappeared.

After Phase 7, the browser remembers those arrays. The UI is rebuilt from saved data when the page starts.

localStorage is not a database, not the cloud, and not your app state. It is a notebook the browser keeps for this site.

The one idea

JavaScript arrays are still the source of truth while the page is open.

localStorage is where those arrays are remembered between refreshes.

```
CHANGE DATA  ->  SAVE  ->  RENDER  
PAGE START    ->  LOAD  ->  RENDER
```

2. What localStorage is

localStorage is a key-value store built into the browser. You save a string under a key, and later you read that same key.

It survives refresh and later visits to the same site, in the same browser, on the same computer. It does not sync to other devices.

The four operations

```
localStorage.setItem(key, value); // save  
localStorage.getItem(key);        // read (null if missing)  
localStorage.removeItem(key);     // delete one key  
localStorage.clear();             // delete ALL keys for this site
```

Do not use clear() in this project. It would wipe everything. Use removeItem if you need to reset one dataset.

Console experiment

```
localStorage.setItem("phase7Test", "hello");  
console.log(localStorage.getItem("phase7Test")); // hello  
localStorage.removeItem("phase7Test");  
console.log(localStorage.getItem("phase7Test")); // null
```

3. Why JSON exists

localStorage only stores strings. If you pass an object or array directly, the browser turns it into a useless string like [object Object].

JSON is text that can represent objects and arrays. Two methods convert back and forth:

- JSON.stringify(data) - JavaScript value to string (before save)
- JSON.parse(string) - string back to JavaScript (after load)

```
const tasks = [{ id: 1, title: "Study JS", completed: false }];  
localStorage.setItem("tasks", JSON.stringify(tasks));  
const loaded = JSON.parse(localStorage.getItem("tasks"));  
console.log(loaded[0].title); // Study JS
```

4. Your storage keys

Three separate keys. One problem in notes does not destroy tasks or events.

tasks -> array of task objects
notes -> array of note objects
events -> array of calendar event objects

Never use a vague key like data. Keep names matching the arrays.

5. Script order in Fesche.html

Helpers must exist before the other files call them. That is why localStorage.js is first.

```
<script src="localStorage.js"></script>
<script src="task-manager.js"></script>
<script src="notes.js"></script>
<script src="calc.js"></script>
```

Classic script tags share one page. saveTasks can see the tasks array even though that array is declared in task-manager.js. The function looks up tasks at call time, not at definition time.

6. localStorage.js - line by line

saveData(key, data)

```
function saveData(key, data) {
  localStorage.setItem(key, JSON.stringify(data));
}
```

Takes a key name and any JavaScript value (your array). Turns the array into a JSON string, then stores it. One helper for all three datasets.

loadData(key, fallback)

```
function loadData(key, fallback) {
  const stored = localStorage.getItem(key);
  if (!stored) {
    return fallback;
  }
  try {
    const parsed = JSON.parse(stored);
    if (Array.isArray(parsed)) {
      return parsed;
    }
    return fallback;
  } catch (error) {
    console.error("Could not load " + key + ":", error);
    return fallback;
  }
}
```

Step 1: getItem. If the key was never saved, stored is null. Return fallback (always [] in this app).

Step 2: try JSON.parse. Bad text in DevTools would crash the whole page without try/catch. On failure, log the error and return [].

Step 3: Array.isArray. JSON.parse('{}') succeeds but gives an object, not a list. Tasks/notes/events must be arrays, so reject anything else.

The six small wrappers

```
function saveTasks() { saveData("tasks", tasks); }
function loadTasks() { tasks = loadData("tasks", []); }

function saveNotes() { saveData("notes", notes); }
function loadNotes() { notes = loadData("notes", []); }

function saveEvents() { saveData("events", events); }
function loadEvents() { events = loadData("events", []); }
```

save* writes the current array. load* replaces the current array with whatever was stored, or [] if nothing valid exists.

Why wrappers? So task-manager.js never mentions localStorage.setItem. It only says saveTasks() after the array changes.

7. Tasks (task-manager.js)

The array starts empty. Sample tasks were removed so storage, not hardcoded data, owns the list.

```
let tasks = [];
```

Startup

```
loadTasks();
renderTasks();
```

Load first, then draw. If you render first you flash an empty list, then overwrite it.

Worse: if you never load, you always start from [].

Add

```
tasks.push({
  id: Date.now(),
  title: title,
  completed: false
});
saveTasks();
renderTasks();
```

id: Date.now() is a unique number from the current time. completed starts false. Then save, then redraw.

Delete

```
tasks = tasks.filter(function (task) {
  return task.id !== id;
});
saveTasks();
renderTasks();
```

filter keeps every task whose id is not the one you deleted. Then save the new array.

Complete checkbox

```
task.completed = checkbox.checked;
saveTasks();
renderTasks();
```

The object already lives inside the tasks array, so changing task.completed changes the array. Save writes that change.

Important: do not only remove the from the DOM. If the array still has the task, refresh brings it back.

8. Notes (notes.js)

```
let notes = [];
let selectedNoteId = null;
```

notes is persisted. selectedNoteId is only which card is open right now. It is not saved.

After refresh, no note is selected until you click one. That is Phase 7, not a bug.

Create

```
notes.push(newNote);
selectedNoteId = newNote.id;
saveNotes();
```

```
renderNotes();
renderEditor();
```

Edit (saveNote)

```
note.title = title;
note.content = content;
note.updatedAt = Date.now();
saveNotes();
renderNotes();
renderEditor();
```

You mutate the object already in the array, then save the whole notes array.

Delete

```
notes.splice(index, 1);
selectedNoteId = null;
saveNotes();
renderNotes();
renderEditor();
```

splice removes 1 item at that index. Then clear selection so the editor goes back to empty state.

Startup

```
loadNotes();
renderNotes();
renderEditor();
```

9. Calendar (calc.js)

```
let events = [];
```

currentDate and selectedDate are not saved. The calendar still opens on today. Only the events array is persisted.

Add

```
events.push(newEvent);
saveEvents();
renderCalendarGrid();
renderEvents();
```

newEvent has id, title, date (YYYY-MM-DD from the selected day), and time.

Delete / edit

deleteEvent splices by id then saveEvents. editEvent changes event.title then saveEvents. Both re-render the grid and the list.

Startup

```
loadEvents();
renderCalendar();
renderEvents();
```

10. Data shapes

Task

```
{ id: 1738..., title: "Study JavaScript", completed: false }
```

Note

```
{ id, title, content, createdAt, updatedAt }
```

Event

```
{ id, title, date: "2026-10-05", time: "10:00" }
```

date is a string key, not a Date object. JSON can store strings and numbers easily. Date objects would not survive stringify/parse the way you expect.

11. How to inspect storage

Chrome / Edge: DevTools -> Application -> Storage -> Local Storage -> your site.

You should see three keys. Values look like JSON arrays, not JavaScript objects.

- Add a task. Open Local Storage. Confirm the tasks string grew.
- Refresh. Confirm the task is still on screen.
- Delete it. Confirm the stored array shrank.
- Repeat for notes and events.

When the UI looks wrong, check both sides: the array in the Console, and the JSON in Application.

12. Common bugs (and how this code avoids them)

- Saving an object without stringify. Fix: saveData always stringifies.
- Forgetting parse. Fix: loadData always parses.
- Save but no render. Fix: every mutation calls save then render.
- Render before load. Fix: loadTasks/loadNotes/loadEvents run first.
- Delete from DOM only. Fix: delete from the array, then save.
- JSON.parse crash on startup. Fix: try/catch + fallback [].
- Stored value is {} not []. Fix: Array.isArray check.

13. What this is NOT

Not a backend. Not login. Not Firebase. Not IndexedDB. Not Phase 8 (linking tasks, notes and calendar together).

Data lives in one browser profile. Clearing site data wipes it. Another computer will not see it.

14. Mental model to memorize

USER clicks / types

- > change tasks / notes / events
- > saveData (stringify + setItem)
- > render UI

PAGE reload

- > loadData (getItem + parse)
- > put result into the array
- > render UI

Four lines:

```
JSON.stringify(data)    // before saving
JSON.parse(string)      // after loading
localStorage.setItem()  // save
localStorage.getItem()  // load
```

If you can add three tasks, two notes and two events, refresh, and everything comes back, Phase 7 is done.